

PDN Abstraction Flow Comparison

Pixel-R (spec_flow) · Tri-stagger · Tri-square (my_flow)
Focus: model topology differences (pitch + IR objective matched)

1. Overview

Both packages start from IBM power-grid SPICE, discover VDD connected components, assemble a sparse mesh conductance $\mathbf{G}_M$, and Kron-reduce internals to a dense port matrix $\mathbf{G}'$. They then stamp a compact R-model and fit parameters so the model matches Kron IR (or, for Pixel-R by default, the port spectrum). The three models below are the primary comparison arms for reduced-vs-model IR plots.

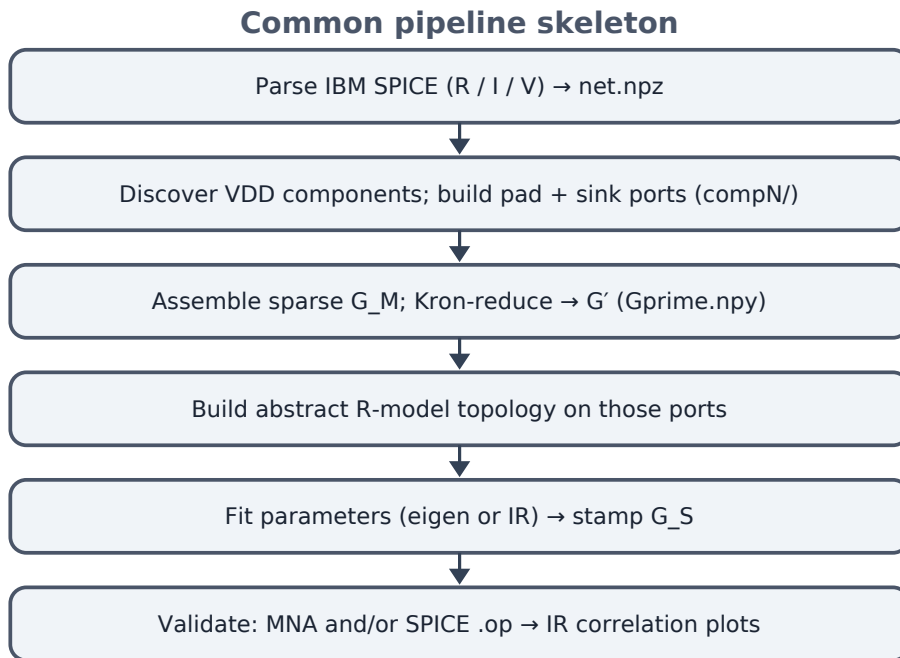
Comparison setup (for now): my_flow uses the **same auto pitch rule** as Pixel-R (pad-array target, clamped > max metal) and the **same IR objective** (absolute multi-stimulus drop match). Differences to study are **model topology and port attach**: Pixel-R star on imaginary grid → **tri-stagger** (real ports + stubs + mid square; 3R) → **tri-square** (L2/L3 hierarchy + stubs; 6R).

my_flow attach rules (ports & vias)

Sink XY = current-weighted average of real SPICE injections in each pitch_bot cell. Pad XY = mean of real V sources in the cell. Then:

- **Voltage pads & current sinks (my_flow):** port = $R_z(/R_{z_pad})$ = landing @ real XY = stubs ($\propto$ sheet R_x/R_y) = nearest mesh node.
- **Tri-stagger:** only those pad/sink vias (no separate sheet-to-sheet via stack).
- **Tri-square:** pads → L2 (stubs $\propto R_{x_u}/R_{y_u}$); sinks → L3 (stubs $\propto R_{x_l}/R_{y_l}$); mid→bot: L2 grid = R_{z_ul} = L3 landing @ L2 XY = stubs on L3 ($\propto R_{x_l}/R_{y_l}$, pitch_l).
- **Pixel-R:** imaginary aligned pad/sink at cell centers; no stub attach (half-arm star cells).

2. Shared front-end (all models)

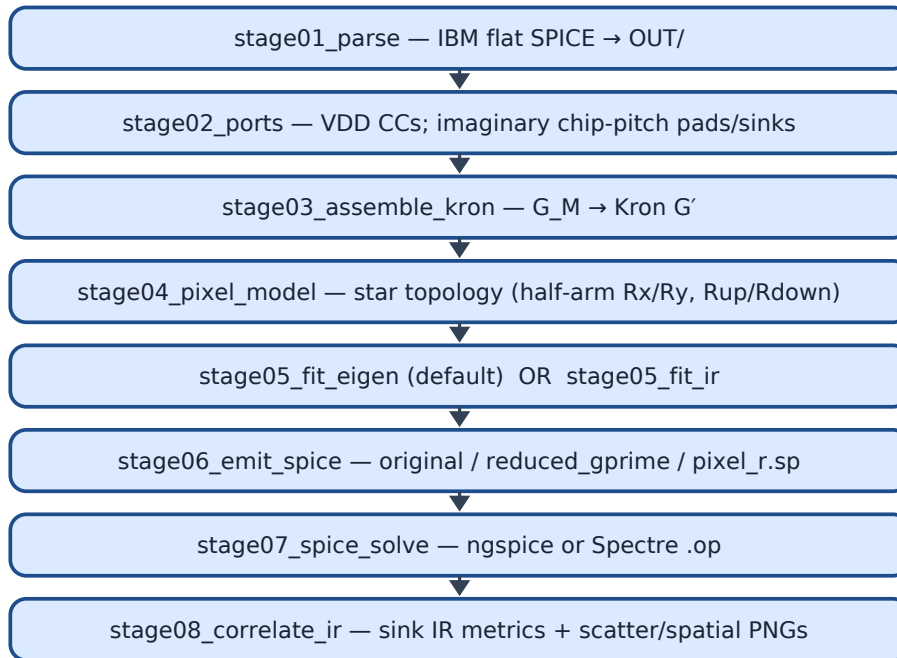


IR definition: $IR_k = \text{mean}(V_{pads}) - V_{sink,k}$ | Mixed BC: pads V-fixed, sinks I-driven

3. Pixel-R flow (spec_flow)

Baseline TSMC Spec abstraction: a **single-layer grid of identical stars**. No geometry extract — three globals (R_x , R_y , R_z) are fit so the Schur port admittance G_s matches Kron G' .

Pixel-R generation flow



Driver: `tclsh spec_flow/run_flow.tcl [ibmpgn] [eigen|ir] | chip_pitch > max metal pitch`

3b. Pixel-R model & fitting

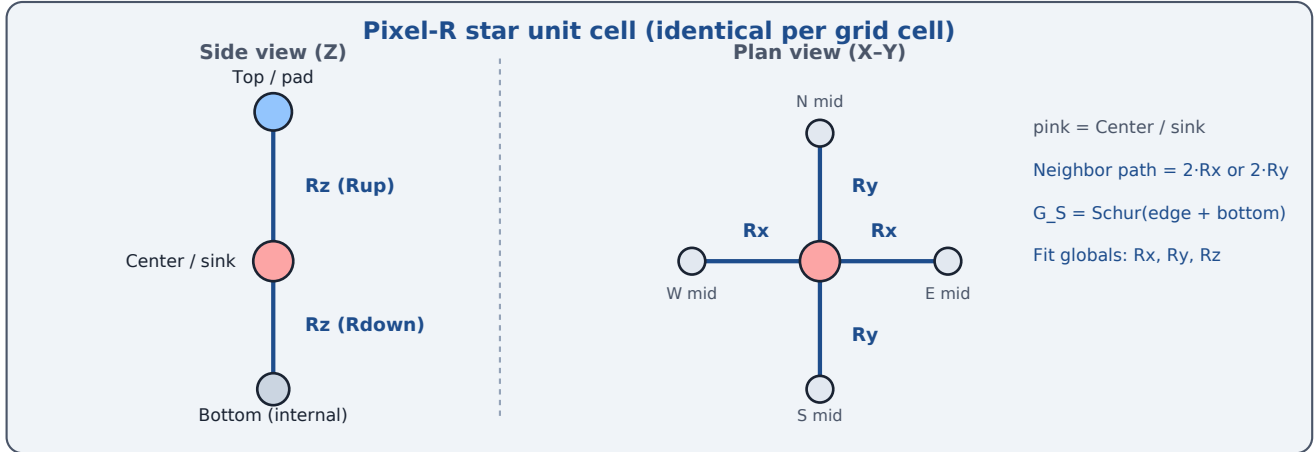


Figure: Pixel-R star. Left = vertical pad-sink-bottom stack; right = in-plane half-arms. Neighbor sinks couple through two series half-arms (effective $g = 1/(2R_x)$ or $1/(2R_y)$).

Fitting details

Eigen objective (default, TSMC spec)

$$\min_{R_x, R_y, R_z} \| \lambda_M - \text{diag}(Q^T G_S Q) \|^2$$

$$G' = Q \text{diag}(\lambda_M) Q^T$$

Grounded eigh of Kron G' (match projected spectrum of G_S to λ_M)

IR objective (alternative)

$$\min_{R_x, R_y, R_z} \sum_j \| V_S(I_j) - V_M(I_j) \|^2$$

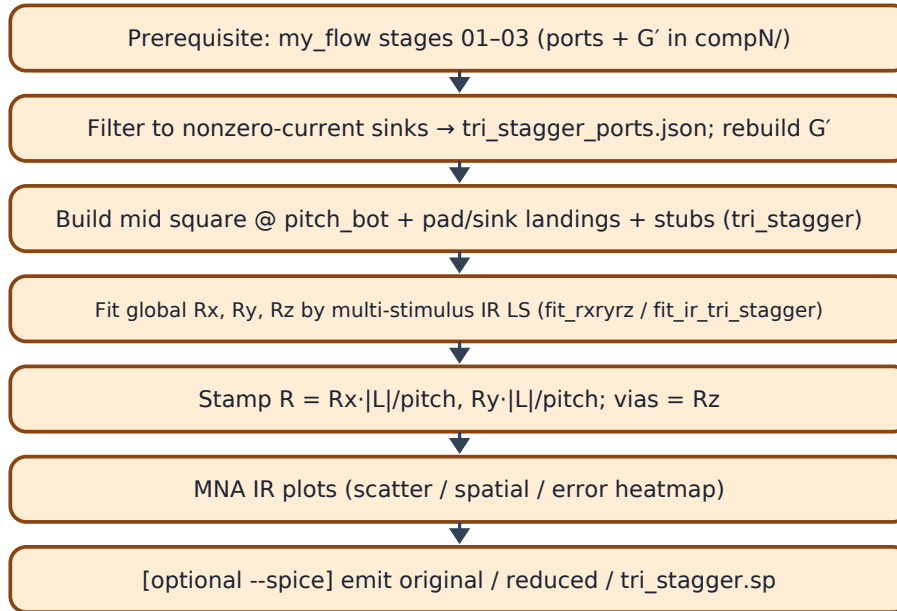
On sink IR drops; multi-stimulus mixed BC (pads V-fixed, sinks I-driven)

- **Optimizer:** `scipy.optimize.least_squares`, method=TRF, variables in $\log_{10}(R)$, bounds $\sim [10^{-4}, 10^4] \times r_{\text{scale}}$.
- **Limitation addressed by my_flow models:** not metal-faithful; aligned pad↔sink; three globals only — hard to capture heterogeneous / multi-layer anisotropy.

4. Tri-stagger flow (my_flow)

Reuses my_flow **OUT/compN** (parse → ports → Kron already done). Builds a **square mid grid** at pitch_{bot}. Attach rule: pad/sink – Rz – landing @ real XY – stubs (∝ Rx/Ry) – nearest mid node. Sink XY is the current-weighted injection average in the cell. No sheet-to-sheet via beyond those pad/sink vias (so pad→sink cannot be a pure vertical short).

Tri-stagger generation flow



Driver: `python3 my_flow/run_tri_stagger.py <compN> [seed] [--spice] · pad-only comps skipped`

Tri-stagger topology

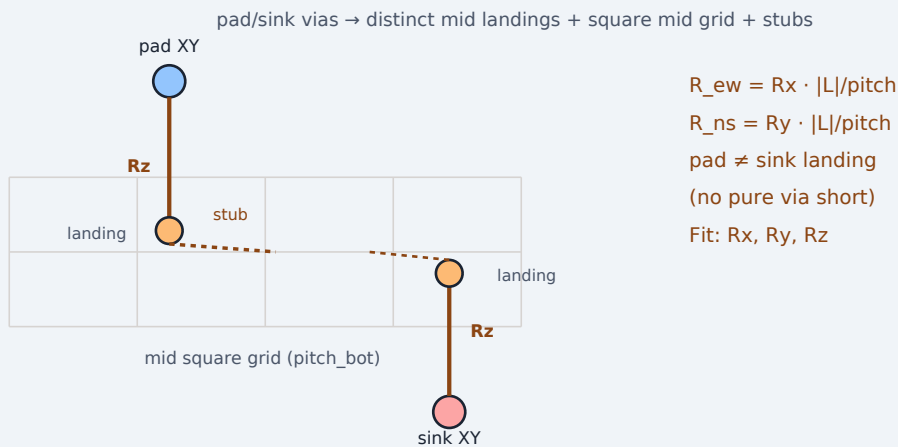


Figure: staggered vias. Distinct mid landings + length-proportional stubs force current through the mid sheet.

Fitting details

IR fit objective (= spec_flow Pixel-R IR)

$$\min_{R_x, R_y, R_z} \sum_j \|\text{drop}_S(I_j) - \text{drop}_M(I_j)\|^2$$

Absolute voltage drops; all stimuli (incl. random); mixed BC

Parameters & optimizer

$$R_{\text{ew}} = R_x \cdot \frac{|L|}{\text{pitch}}, \quad R_{\text{ns}} = R_y \cdot \frac{|L|}{\text{pitch}}, \quad R_{\text{via}} = R_z$$

TRF on $\log_{10}(R_x, R_y, R_z)$; bounds from $\text{median}(\text{diag } G')$; same as `fit_ir.py`

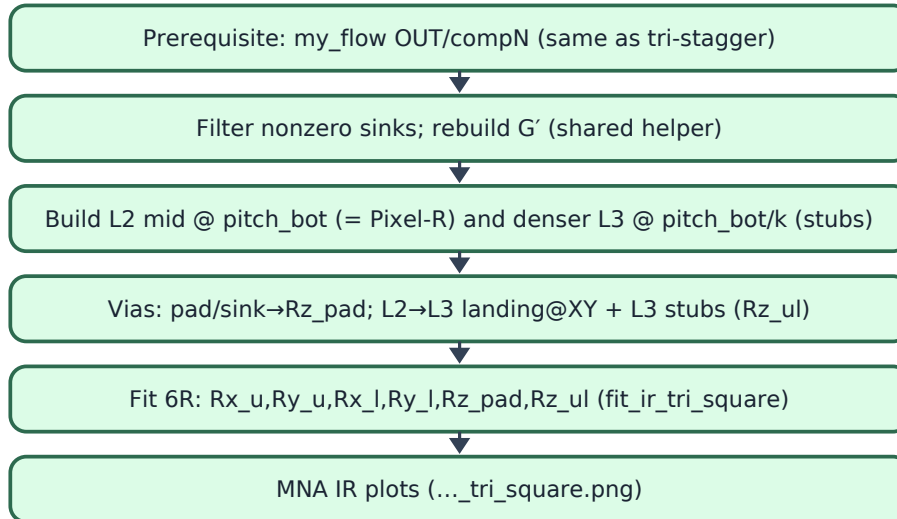
What changes vs Pixel-R (model focus)

- Pitch + IR objective matched to Pixel-R IR — isolate topology.
- Ports: port—Rz—landing—stubs→mesh (vs imaginary aligned ports).
- Full square mid sheet (E-W and N-S) vs Pixel-R abstract star cells.
- Distinct landings so pad→sink cannot be a pure via short.
- Same globals R_x, R_y, R_z ; stub $R \propto R_x/R_y$, via = R_z .

5. Tri-square flow (my_flow)

Same my_flow **OUT/compN** prerequisite and port filtering as tri-stagger, but stamps **two** square sheets with **six** R parameters: mid L2 at the **Pixel-R pitch** ($\text{pitch}_{\text{bot}}$) and denser bot L3 ($\text{pitch}_l = \text{pitch}_{\text{bot}}/k$). **Pads:** $V - R_{z_pad} - L2 \text{ landing} - \text{stubs} (\propto R_{x_u}/R_{y_u}) \rightarrow L2$. **Sinks:** $I - R_{z_pad} - L3 \text{ landing} - \text{stubs} (\propto R_{x_l}/R_{y_l}) \rightarrow L3$. **Mid→bot vias:** $L2 \text{ grid} - R_{z_ul} - L3 \text{ landing @ } L2 \text{ XY} - \text{stubs} (\propto R_{x_l}/R_{y_l}, \text{pitch}_l) \rightarrow L3 \text{ grid}$.

Tri-square generation flow



Driver: `python3 my_flow/run_tri_square.py <compN> [seed] [-k 2] · default k=2 (L3 denser; pitch_l = pitch_bot/k)`

Tri-square stub stack (6 R: per-sheet Rx,Ry + 2 vias)

L1 pads → L2 (coarse) → L3 (fine) → sinks

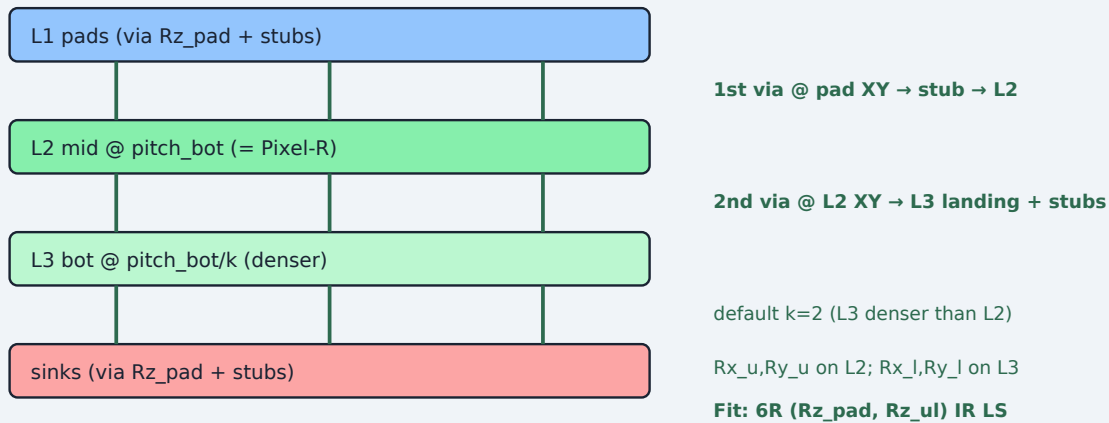


Figure: dual-pitch sheets. L2 @ $\text{pitch}_{\text{bot}}$ (= Pixel-R), L3 denser. Six R params: per-sheet Rx/Ry plus R_{z_pad} (ports) and R_{z_ul} (L2→L3).

Fitting details

IR fit objective (= spec_flow Pixel-R IR)

$$\min_R \sum_j \|\text{drop}_S(I_j) - \text{drop}_M(I_j)\|^2$$

Same absolute-drop residual as stagger / Pixel-R IR

Parameters & optimizer (6)

$$R_x^u, R_y^u, R_x^l, R_y^l, R_z^{\text{pad}}, R_z^{\text{ul}}$$

TRF on $\log_{10}(R)$; L2 vs L3 differ by pitch and independent Rx/Ry

Improvements vs tri-stagger

- Two sheets: L2 mid @ Pixel-R pitch + denser L3 (pitch_bot/k).
- Six R params: Rx_u, Ry_u, Rx_l, Ry_l, Rz_pad, Rz_ul (not shared 3-globals).
- Pads/sinks: port—Rz_pad—landing—stubs→sheet.
- Mid→bot vias: L2—Rz_ul—L3 landing@XY—stubs→L3 grid (not pure via).
- Bilinear attach retired.

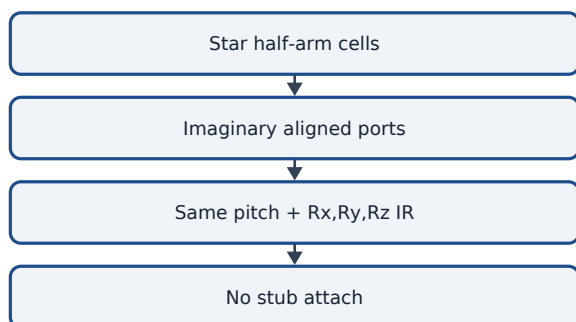
6. Side-by-side comparison

Pitch policy and IR fitting are matched to Pixel-R IR. The remaining differences are **model structure**: star vs mid square vs L2/L3, and imaginary aligned ports vs real XY + stubs.

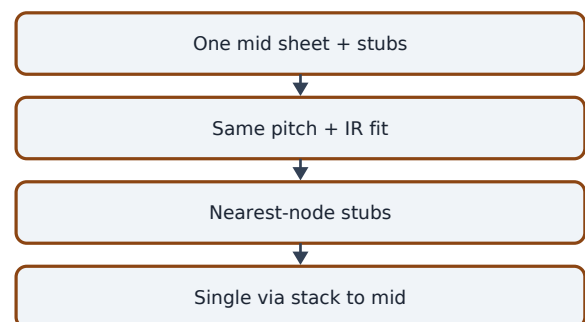
Aspect	Pixel-R (spec_flow)	Tri-stagger (my_flow)	Tri-square (my_flow)
Role / lineage	TSMC Spec baseline star abstraction	my_flow: real ports + stubs + global Rx,Ry,Rz (Pixel-R-style fit)	Same ports/IR objective; L2/L3 + 6 R params
Topology tag	star_half_arm	tri_stagger_mid_grid	tri_square_stub_6r
Metal / sheets	Single abstract star grid (not physical metals)	One mid square sheet + pad/sink landings + stubs	L2 coarse + L3 fine (per-sheet Rx,Ry)
Pitch (auto)	Pad-array target, clamp > max metal	Same rule as Pixel-R (shared for fair model compare)	Same as stagger / Pixel-R
Port model	Imaginary full chip-pitch grid; $n_{\text{pads}} = n_{\text{sinks}} = n_x \times n_y$	Real pad XY + real nonzero-current sinks (may $n_p \neq n_s$)	Same as stagger (reuse tri_stagger_ports_from_dual)
Lateral R	Global half-arms Rx, Ry (identical cells)	$R = R_x \cdot L /\text{pitch}$ (EW), $R_y \cdot L /\text{pitch}$ (NS) on mid + stubs	L2: R_{x_u}/R_{y_u} ; L3: R_{x_l}/R_{y_l} (each $\cdot L /\text{pitch_sheet}$)
Vertical / via	Global Rz ($R_{\text{up}} = R_{\text{down}} = R_z$); no stubs	Pad/sink: Rz then stubs; no sheet-sheet via	Rz_pad (pad/sink); Rz_ul (L2→L3) then L3 stubs
Port / via attach	Pad & sink at same cell center (aligned)	port—Rz—landing—stubs→mid grid	port—Rz_pad—landing—stubs; L2—Rz_ul—L3 landing—stubs
R extract	None — fit R directly from Kron G'	None — fit R directly (Zhang extract retired)	None — fit R directly (Zhang extract retired)
Fit variables	Rx, Ry, Rz (3 globals)	Rx, Ry, Rz (3 globals)	$R_{x_u}, R_{y_u}, R_{x_l}, R_{y_l}, R_{z_pad}, R_{z_ul}$ (6)
Fitting objective	Default: eigen LS $\ \lambda_M - \text{diag}(Q^T G_S Q) \ ^2$; alt: IR voltage match	Same as Pixel-R IR: $\Sigma \ \text{drop}_S - \text{drop}_M \ ^2$ (fit_rxyzr)	Same Pixel-R IR objective (6-param TRF)
Fitting method	scipy least_squares TRF; $\log_{10}(R)$; $\text{max_nfev} \approx 300$	Same TRF on $\log_{10}(R_x, R_y, R_z)$ as Pixel-R IR	TRF on $\log_{10}$ of 6 R params (fit_log_r_params_ir)
BC / residual	Eigen: spectrum of G' ; IR: pads V-fixed, sinks I-driven	Mixed BC; absolute drop residual (all stimuli)	Same mixed-BC IR residual as stagger / Pixel-R IR
SPICE validation	Always emit + solve original / reduced / pixel_r	Optional --spice; MNA IR plots always	MNA IR plots always
Entry point	tclsh spec_flow/run_flow.tcl [case] [eigen ir]	run_tri_stagger.py <comp> [seed] [--spice]	run_tri_square.py <comp> [seed] [--k 2]
Key artifacts	pixel_model.json, pixel_r.json, spice/pixel_r.sp	tri_stagger_model.json, tri_stagger_r.json (Rx,Ry,Rz)	tri_square_model.json, tri_square_r.json (6R)

7. What each step improves

Pixel-R → Tri-stagger



Tri-stagger → Tri-square



Pixel-R → Tri-stagger: change topology/ports (star → mid square + real stubs); keep pitch + IR objective.
Tri-stagger → Tri-square: add L2/L3 hierarchy and expand to 6 R params.

From → To	Primary improvement	Fitting / model change
Pixel-R → Tri-stagger	Mid square + real ports + stubs (model change)	Pitch + IR objective held fixed (= Pixel-R IR)
Tri-stagger → Tri-square	L2/L3 hierarchy; per-sheet Rx/Ry + 2 vias	6R fit (Rx_u,Ry_u,Rx_l,Ry_l,Rz_pad,Rz_ul); same IR objective
Shared validation	Correlate sink IR vs Kron G' (and vs original SPICE when solved)	$e_{IR} = \Delta V_{sink} / V_{ref} $; plots under each compN/

8. How to reproduce the comparison plots

Fair compare for **model** diffs: same auto pitch (> max metal) and same IR objective (fit_rxryrz = Pixel-R IR).

- 1. my_flow (stages 01-03 + both models): tclsh my_flow/run_flow.tcl ibmpg2
 - 2. Or per-comp after stages 01-03: python3 my_flow/run_tri_stagger.py my_flow/outputs/ibmpg2_auto_k1/comp1 0 · python3 my_flow/run_tri_square.py ../comp1 0 --k 2
 - 3. Pixel-R IR arm: tclsh spec_flow/run_flow.tcl ibmpg2 ir
- Compare ir_scatter_reduced_vs_*.png under each compN/ (ibmpg2-6).

Sources: spec_flow/{pixel_r,fit_eigen,fit_ir,stage0*}.py · my_flow/{stub_attach,fit_rxryrz,tri_stagger,tri_square,run_*.py · READMEs